

MUGO JESCINTA NYAMBURA

CIT-223-013/2024

FOUNDATIONS OF AI

Cat 1

### **Question One (15 marks)**

a) Describe the concept of Search indicating major differences between informed search and uninformed search.

In AI, Search is the process of navigating through a "state space" to find a sequence of actions that leads from an initial state to a goal state.

#### **Differences**

Knowledge:

- Uninformed Search (Blind Search) Informed Search (Heuristic Search) Knowledge - No domain-specific knowledge; only knows how to generate successors and recognize the goal.
- Informed Search- Uses domain-specific hints (heuristics) to estimate the distance to the goal.

Efficiency :

- Uninformed Search- Often slow and explores many unnecessary nodes. Highly
- Informed Search- Highly efficient; targets the goal directly.

Examples-

- Uninformed- Breadth-First Search (BFS), Depth-First Search (DFS).
- Informed- A\* Search, Greedy Best-First Search.

b) Write a program in python to implement A\* Search Strategy that optimizes the path resources.

A\* optimizes path resources by minimizing  $f(n) = g(n) + h(n)$ , where  $g(n)$  is the cost to reach the node and  $h(n)$  is the estimated cost to the goal.

```
import heapq
```

```
def a_star_search(graph, start, goal, heuristics):
```

```
    # priority_queue stores (f_score, current_node, path, actual_cost)
```

```

priority_queue = [(heuristics[start], start, [start], 0)]
visited = set()
while priority_queue:
    f, current, path, g = heapq.heappop(priority_queue)
    if current in visited:
        continue
    visited.add(current)
    if current == goal:
        return path, g
    for neighbor, weight in graph.get(current, {}).items():
        if neighbor not in visited:
            new_g = g + weight
            new_f = new_g + heuristics.get(neighbor, 0)
            heapq.heappush(priority_queue, (new_f, neighbor, path + [neighbor], new_g))
return None, float('inf')

# Example Usage
graph = {
    'A': {'B': 1, 'C': 3},
    'B': {'D': 5},
    'C': {'D': 1},
    'D': {}
}
heuristics = {'A': 4, 'B': 2, 'C': 1, 'D': 0}
path, cost = a_star_search(graph, 'A', 'D', heuristics)
print(f"Optimal Path: {path} with cost: {cost}")

```

c) (i) Distinguish the following concepts: valid, satisfiable, unsatisfiable and falsible.

- Valid (Tautology): True for every possible assignment of truth values.
- Satisfiable: True for at least one assignment of truth values.
- Unsatisfiable (Contradiction): False for every possible assignment.
- Falsible: False for at least one assignment.

(ii) Using Truth Tables describe where the following formulae are valid, satisfiable, unsatisfiable, or falsible. Indicate under what conditions will each be attained.  $P \wedge Q \rightarrow \neg R$

| P | Q | R | P and R | -R | P and R $\rightarrow$ -R |
|---|---|---|---------|----|--------------------------|
| T | T | T | T       | F  | F                        |
| T | T | F | T       | T  | T                        |
| T | F | T | F       | F  | T                        |
| T | F | F | F       | T  | T                        |
| F | T | T | F       | F  | T                        |
| F | T | F | F       | T  | T                        |
| F | F | T | F       | F  | T                        |
| F | F | F | F       | T  | T                        |

Conclusion:

Satisfiable: Yes (True in 7 rows).

Falsible: Yes (False when P, Q, R are all True).

Valid: No (It is not True in all cases).

Unsatisfiable: No (It is not False in all cases).

### **Question Two (10 marks)**

- a) The Ministry of Health in consultation with the Ministry of Interior have asked MMU Artificial Intelligence Institute to develop agent to automate DNA and Genomic Sequencing of all Shakahola victims for faster identification based on samples harvested from corpses and their relatives. You are to come with the PEAS and PAGE

of the system. Outline possible PEAS and PAGE elements of the Automatic DNA and Genomic Sequencing System.

For a system identifying victims via DNA sequencing:

PEAS (Performance, Environment, Actuators, Sensors)

- Performance: Accuracy of DNA matches, processing speed, data security.
- Environment: Forensic labs, databases (Ministry of Interior records), morgues.
- Actuators: Display screen (for results), report generators, DNA sequence assemblers.
- Sensors: DNA scanners, biometric input, database query interfaces.

PAGE (Percepts, Actions, Goals, Environment)

- Percepts: Raw DNA samples (A, T, C, G sequences), family reference samples.
- Actions: Compare sequences, compute probability of match, flag anomalies.
- Goals: Correctly identify 100% of victims; link relatives to remains.
- Environment: High-stress, large datasets, legally sensitive.

- b) Design and implement a simple Vacuum cleaner agent that cleans the various locations satisfactorily. Clearly comment your work.

This agent follows a simple reflex rule: if the current location is dirty, clean it; otherwise, move to the next location.

class VacuumAgent:

```
def __init__(self, locations_status):  
    # locations_status: {'A': 'Dirty', 'B': 'Clean'}  
    self.locations = locations_status  
  
def act(self, current_location):  
    print(f"Agent is at Location {current_location}")  
    # Rule 1: If Dirty, Suck (Clean)  
    if self.locations[current_location] == 'Dirty':  
        print(f"--- Location {current_location} is Dirty. Cleaning...")  
        self.locations[current_location] = 'Clean'
```

```

        print(f"--- Location {current_location} is now Clean.")
    else:
        print(f"--- Location {current_location} is already Clean.")

    # Rule 2: Move to the other location
    next_location = 'B' if current_location == 'A' else 'A'
    print(f"--- Moving to Location {next_location}.\n")
    return next_location

# Practical Execution
environment = {'A': 'Dirty', 'B': 'Dirty'}
agent = VacuumAgent(environment)
loc = 'A'

for _ in range(2): # Run for two steps to check both rooms
    loc = agent.act(loc)

```

### Explanation

The agent uses a Reflexive Mechanism. It doesn't need to know the history of the entire world; it only looks at its current percept (Location and Status) to decide the next action. This is the most basic form of intelligence in agent design.